Simple Maven Project

Introduction to Maven

Apache Maven is an open-source build automation and project management tool mainly used for Java projects. In DevOps, Maven helps developers automate tasks such as compiling code, testing, packaging, and managing project dependencies.

Maven follows a standard project structure and uses an XML file called pom.xml (Project Object Model) to manage project configuration.

Brief Explanation of Maven

Maven simplifies the software development process by automating the build lifecycle. It is widely used in DevOps for Continuous Integration (CI) and Continuous Deployment (CD).

Key Features of Maven

1. **Build Automation**
    - Automatically compiles source code and creates application packages like JAR or WAR files.
2. **Dependency Management**
    - Downloads required libraries from online repositories automatically.
3. **Standard Project Structure**
    - Provides a fixed directory structure, making projects easier to manage.
4. **Easy Integration**
    - Works well with DevOps tools such as:
        - Jenkins
        - Git
        - Docker
5. **Testing Support**
    - Supports automated testing using frameworks like JUnit.

Working of Maven in DevOps

1. Developer writes code.
2. Code is pushed to Git repository.
3. Jenkins triggers Maven build.
4. Maven compiles code and runs tests.
5. Maven packages the application.
6. Application is deployed to the server.

Advantages of Maven

- Simple and easy to use
- Reduces manual work
- Automatic dependency management
- Faster project setup

- Supports CI/CD pipelines

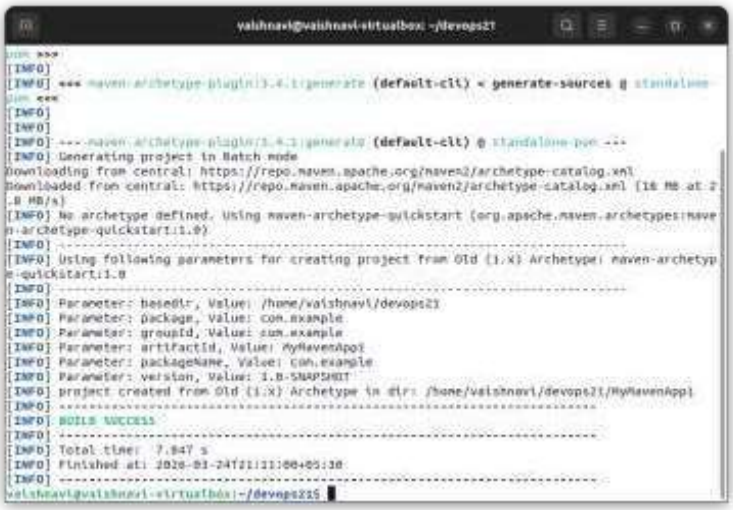
Conclusion

Maven is an important DevOps tool used for automating software builds and managing dependencies. It improves productivity, ensures consistency, and supports continuous integration and deployment in modern software development.

Step1:

To create a basic Maven Java project, run the following command:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false



The command can be broken down as:

DgroupId=com.example – project's base package (reverse-domain style).

-DartifactId=my-maven-app – defines the name of the project (will also be the folder name).

-DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.

-DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use $cd command to go into the directory.

Execute $tree command to view the structure of the project

```
vaishnavi@vaishnavi-virtualbox:~/devops21$ cd MyMavenApp1
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$ tree
.
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── com
    │           └── example
    │               └── App.java
    └── test
        └── java
            └── com
                └── example
                    └── AppTest.java

9 directories, 3 files
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$
```

Steo 3:

Edit the pom.xml file

Add the required dependencies and plugins required from central maven respository

For given project, we will download unit testing dependency from Jenkins.

Step 4:

Execute the command $mvn compile

If the command shoes output as “build success”, it means that the project has compiled successfully

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:MyMavenApp >----------------------
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is plat
form dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.713 s
[INFO] Finished at: 2026-03-24T21:20:16+05:30
[INFO] ------------------------------------------------------------------------
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$
```

Step 5:

Execute $tree command again to view the complete structure after adding plugins and dependencies to the pom.xml file

Step 6:

Execute $mvn test to run unit tests

Step 7:

Run mvn package command to generate a JAR application bundle which contains our basic Maven Hello World application

Step 8:

Run the following command to run the JAR file: java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

```
[INFO] ------------------------------------------------------------------------
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
HelloClass, happy to see you all
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$
```

Pushing The Project into Git:

Execute the commands:

$ git config --global user.name “vaishnavim1121”

$ git config --global user.email “vaishnavim1121@gmail.com”

$git init

$git add .

$git commit -m “initial commit”

$ git remote add origin https://github.com/vaishnavim1121/MyMavenApp.git

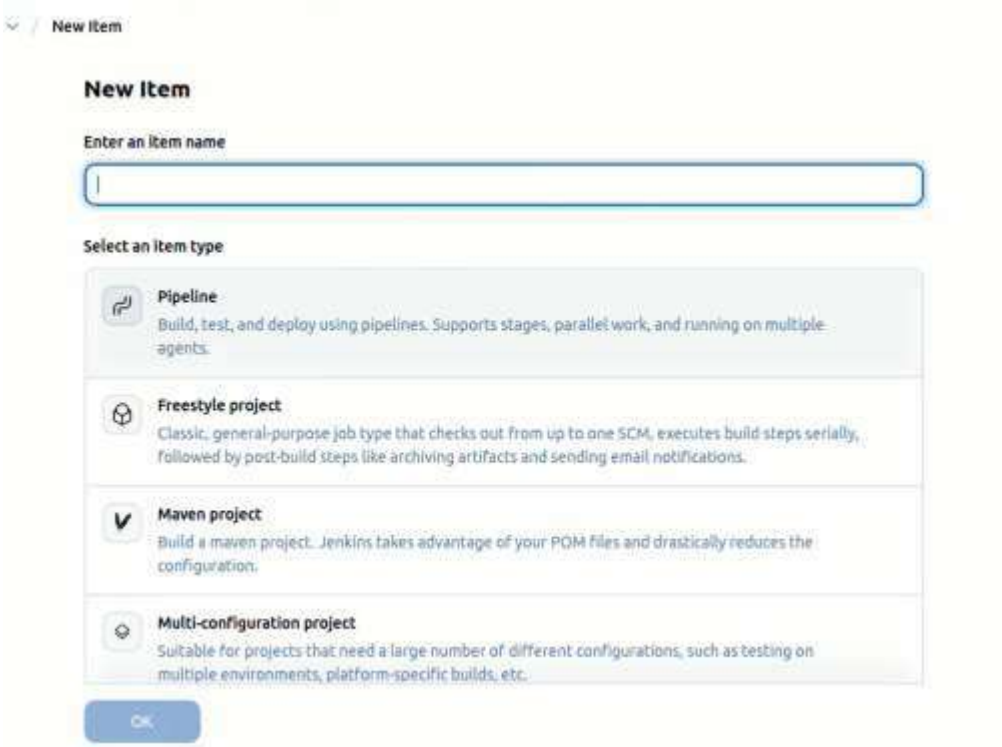
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenApp

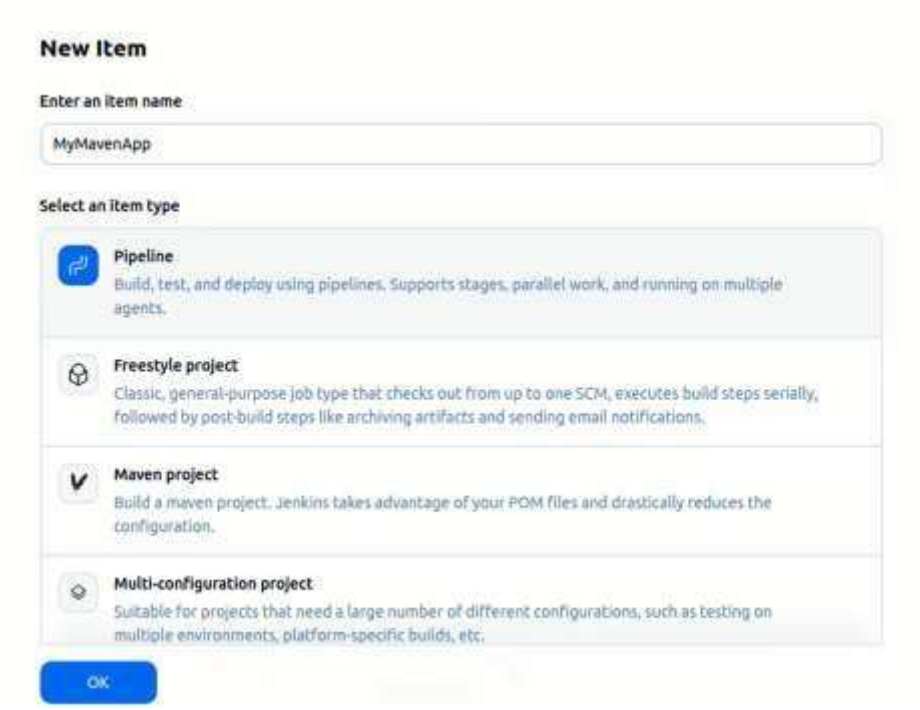
$ git push -u origin master

Second Phase

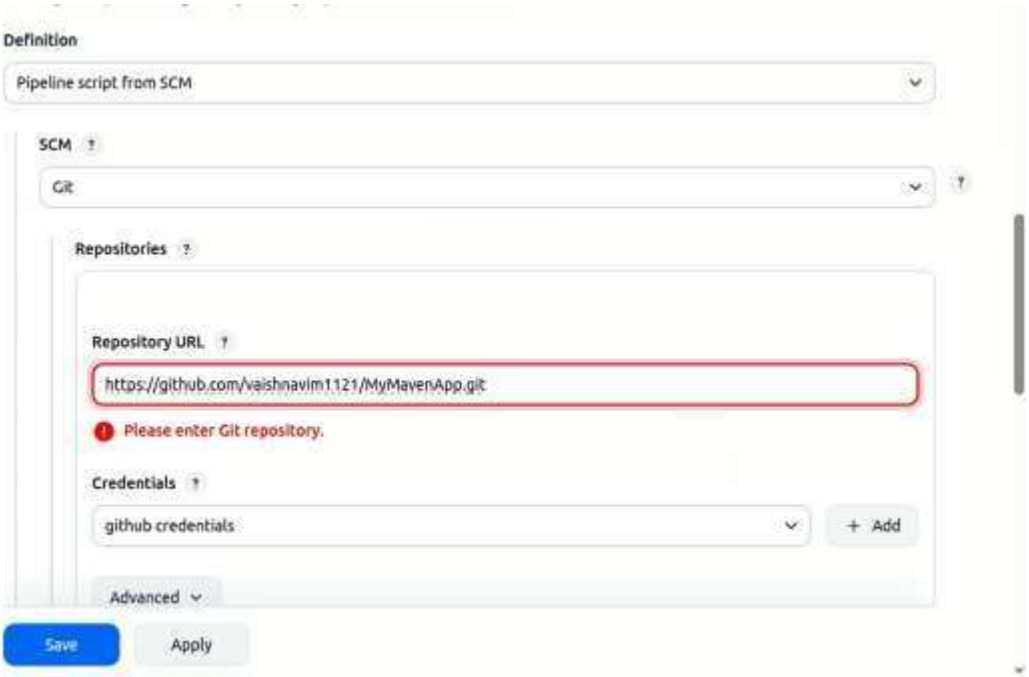
Depicting Jenkins Pipeline
Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository
go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.

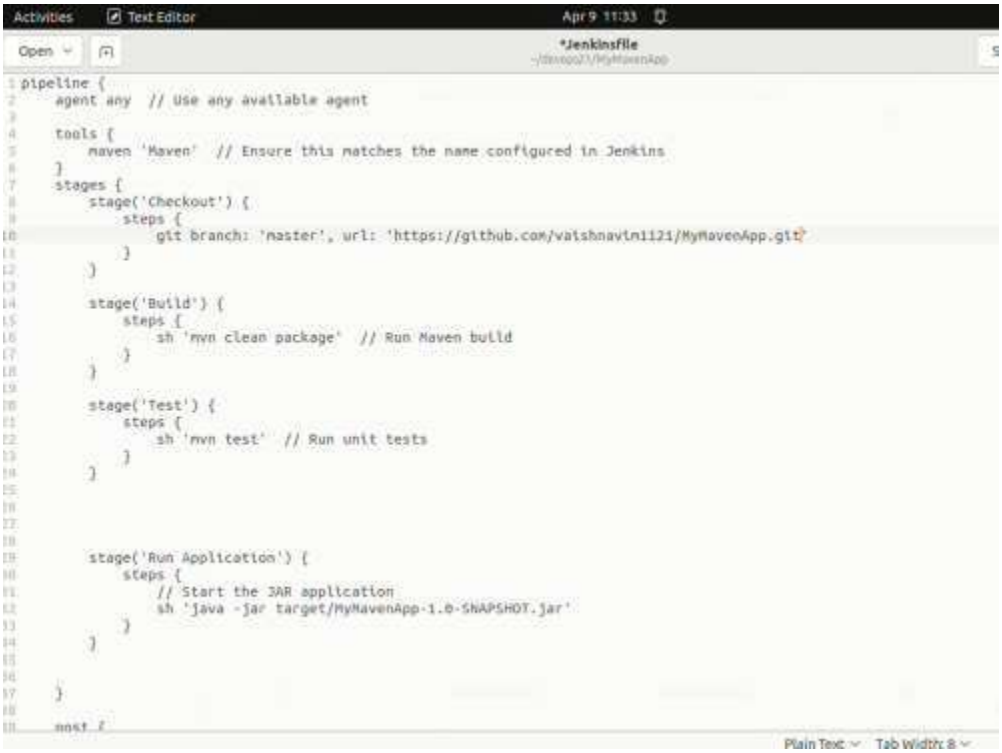




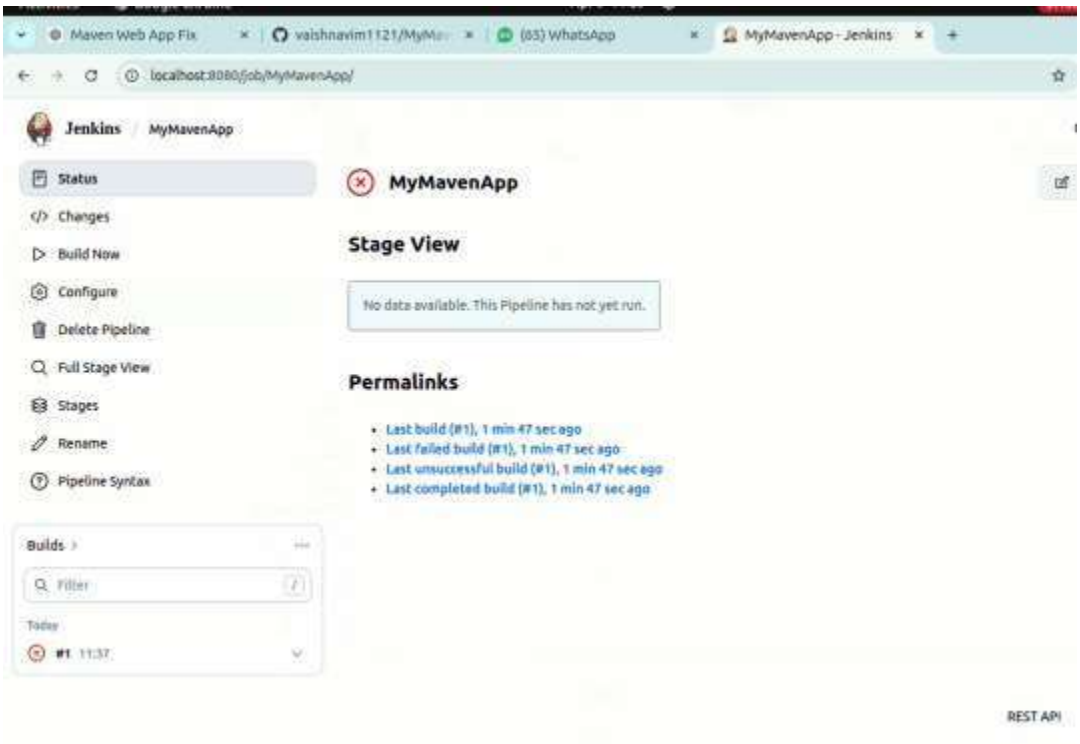
This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page



Now we need to addd a jenkinsfile into out project which will consist of the operations that will occur when building the project in Jenkins
It includes step wise instructions guiding at each stage with right commands



Now try building the project with Jenkins



The error occurs because there is no Jenkinsfile in the github repository. Thus we need to push the content to our project repository in order to run the project successfully.

Push the project again to your git account
Now try building the project in Jenkins by clicking on build-now option.

Check the logs in order to rectify the error.
The error is due to version being outdated,

In order to correct it, edit the version that is mentioned in the pom.xml file.
Change the version from 1.7 to 11 in both souce and target tabs.
After commiting the changes, build again in the Jenkins